

به نام خدا

توضیحات پیاده سازی پروژه پایانی درس مبانی برنامه نویسی کامپیوتر

بهمن 1404

ش.د: 404412439

نام و نام خوانوادگی: محمدرضا غلامی

راه های ارتباطی:

1- @mmd1023 (تلگرام، بله و ...)

2- <https://github.com/mmd1023>

3- mmd.gh313@gmail.com

4- <https://t.me/ElectroGeek255>

عنوان: پروژه 3 (Pentago-X)

 [فایل توضیحات پروژه \(برای دانلود از گیتهاب کلیک کنید\)](#)

مقدمه

این بازی فارغ از پیاده سازی های مربوط به منو، رابط کاربری، ذخیره اطلاعات و دیگر آپشن ها (شامل تایمر و کنسول رنگی)، دو چالش الگوریتمی دارد که به دو تسک جدا و اصلی پروژه تبدیل میشوند.

تسک 1: جاگذاری مهره ها و چرخش زمین بازی

تسک 2: بررسی پایان بازی و تعیین برنده

بنابراین در این مطلب ابتدا دو تسک ذکر شده و سپس آپشن های پیاده سازی شده را شرح خواهیم داد.

در نهایت هم مثالی عملی از کارکرد برنامه نهایی ارائه میشود.

تسک 1

تابع rotate_block:

تابع rotate_block قرار است با ورودی گرفتن اطلاعات زمین بازی (board و n) و موقعیت شروع بلاک انتخاب شده (start_x و start_y) و همچنین جهت چرخش (clockwise)، بلاک مورد نظر را بچرخاند.

```
void rotate_block(unsigned char **board, int n, int start_x, int start_y, bool clockwise) {
    unsigned char temp[n/2][n/2];
    for (int i = 0; i < n / 2; i++)
        for (int j = 0; j < n / 2; j++)
            temp[i][j] = board[start_x + i][start_y + j];

    for (int i = 0; i < n / 2; i++) {
        for (int j = 0; j < n / 2; j++) {
            if (clockwise)
                board[start_x + j][start_y + (n / 2 - 1 - i)] = temp[i][j]; // (i, j) -> (j, n/2-1-i)
            else
                board[start_x + (n / 2 - 1 - j)][start_y + i] = temp[i][j]; // (i, j) -> (n/2-1-j, i)
        }
    }
}
```

بعد از تعریف و مقداردهی آرایه کمکی temp که قرار است کپی ای از مقدار قبل از چرخش آن بلاک باشد، بخش اصلی تابع یعنی چرخاندن ماتریس تحت تبدیل دوران را چنین داریم:

چرخش ساعتگرد ماتریس تحت تبدیل دوران به اندازه 90 درجه:

$$(i, j) \rightarrow (j, n/2-1-i)$$

چرخش پادساعتگرد ماتریس تحت تبدیل دوران به اندازه 90 درجه:

$$(i, j) \rightarrow (n/2-1-j, i)$$

تسک 2: (تابع `count_dir`، `check_win`، `check_win_border` و `check_win_all`)

تابع `count_dir`:

اصل ایده بازگشتی بررسی توالی قرار گیری مهره ها پشت سر هم و در جهات مختلف از دل این تابع نشعت میگیره، این تابع ابزار اصلی برای شمارش توالی مهره ها در هر جهتی و شمارش تعداد آنهاست.

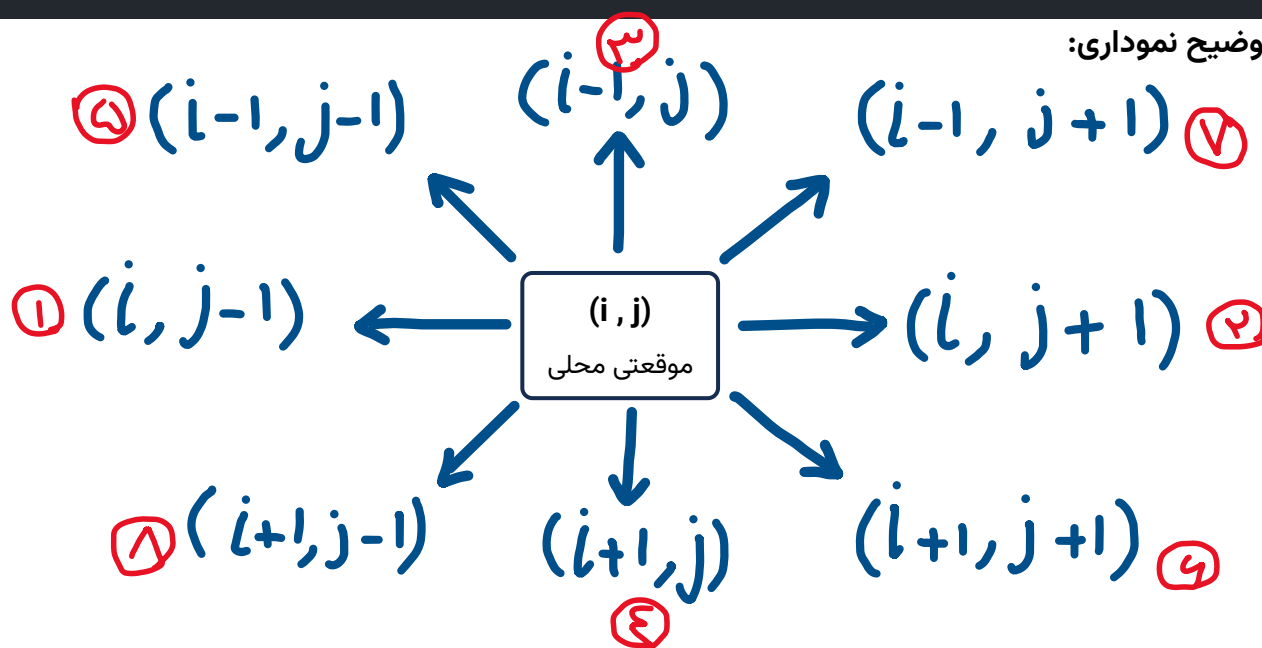
```
int count_dir(unsigned char **borad, int n, int i, int j, char player, int di, int dj, int k, int count = 0) {  
    if (count >= k - 1 || i >= n || i < 0 || j >= n || j < 0 || borad[i][j] != player)  
        return count;  
    return count_dir(borad, n, i+di, j+dj, player, di, dj, k, count + 1);  
}
```

تابع `check_win`:

این تابع با استفاده از تابع `count_dir`، تعداد توالی مهره های هر بازیکن داده شده را می‌شمارد و در صورتی که به حد نصاب بُرد رسیده باشد، `true` برمیگرداند.

```
bool check_win(unsigned char **mat, int n, int i, int j, char player, int k) {  
    int row_score = count_dir(mat, n, i, j-1, player, 0, -1, k) + 1 + count_dir(mat, n, i, j+1, player, 0, 1, k);  
    if (row_score >= k) return true; ❶ ❷  
  
    int col_score = count_dir(mat, n, i-1, j, player, -1, 0, k) + 1 + count_dir(mat, n, i+1, j, player, 1, 0, k);  
    if (col_score >= k) return true; ❸ ❹  
  
    int dia1_score = count_dir(mat, n, i-1, j-1, player, -1, -1, k) + 1 + count_dir(mat, n, i+1, j+1, player, 1, 1, k);  
    if (dia1_score >= k) return true; ❺ ❻  
  
    int dia2_score = count_dir(mat, n, i-1, j+1, player, -1, 1, k) + 1 + count_dir(mat, n, i+1, j-1, player, 1, -1, k);  
    if (dia2_score >= k) return true; ❼ ❽  
  
    return false;  
}
```

توضیح نموداری:



حال کافیتست تابع `check_win` را به ازای هر خانه ای که ممکن است باعث وقوع برد شود صدا کنیم.

ایده اولیه: معمولا برای بازی پنتاگو میایند و تابع `check_win` را به ازای تمام خانه های زمین صدا میزنند که با توجه به ابعاد محدود زمین از لحاظ بهینگی قابل قبول است.

ایده نهایی: میتوان اثبات کرد تنها خانه هایی که با بررسی آنها میتوان ادعا کرد که حتما وقوع برد به درستی چک شده است: 1. خانه ای که در آن مهره جدید گذاشته شده و 2. خانه های مرزی داخلی بلاک چرخیده شده است. مورد 1 که واضح است، اما درمورد مورد 2؛ توجه کنید که خانه های چرخیده شده، تغییر کرده محسوب میشوند و هر تغییری میتواند منجر به برد شود.

اما چرا فقط بررسی خانه های مرزی کفایت میکند؟

اولا فرض میکنیم در حالات قبل هیچ برد ای اتفاق نیوفتاده پس هیچ توالی ای درون بلاک چرخیده شده و در بقیه بلاک ها خود به خود منجر به برد نمیشود. و زنجیره توالی مهره هایی که ممکن است با چرخش منجر به برد شود قطعا زنجیره ای بین بلاکی است، بنابراین، صرفا بررسی خانه های روی مرز داخلی بلاک چرخیده شده واجد شرایط برای ایجاد برد هستند.

تابع `check_win_border`:

با توجه به توضیحات بالا، لزوم پیاده سازی این تابع مشخص شده و کد آن این چنین است:

```
void check_win_border(unsigned char **board, int n, int k, int block_id, bool &first_win, bool &second_win) {
    if (block_id == 1) {
        for (int i = 0; i < n/2; i++) // right border
            if (check_win(board, n, i, n/2-1, board[i][n/2-1], k)) if(board[i][n/2-1] == 'O') first_win = true;
            else if(board[i][n/2-1] == 'X') second_win = true;

        for (int j = 0; j < n/2; j++) // bottom border
            if (check_win(board, n, n/2-1, j, board[n/2-1][j], k)) if(board[n/2-1][j] == 'O') first_win = true;
            else if(board[n/2-1][j] == 'X') second_win = true;
    }
    else if (block_id == 2) {
        for (int i = 0; i < n/2; i++) // left border
            if (check_win(board, n, i, n/2, board[i][n/2], k)) if(board[i][n/2] == 'O') first_win = true; else
            if(board[i][n/2] == 'X') second_win = true;

        for (int j = n/2; j < n; j++) // bottom border
            if (check_win(board, n, n/2-1, j, board[n/2-1][j], k)) if(board[n/2-1][j] == 'O') first_win = true;
            else if(board[n/2-1][j] == 'X') second_win = true;
    }
    else if (block_id == 3) {
        for (int j = 0; j < n/2; j++) // top border
            if (check_win(board, n, n/2, j, board[n/2][j], k)) if(board[n/2][j] == 'O') first_win = true; else
            if(board[n/2][j] == 'X') second_win = true;
        for (int i = n/2; i < n; i++) // right border
            if (check_win(board, n, i, n/2-1, board[i][n/2-1], k)) if(board[i][n/2-1] == 'O') first_win = true;
            else if(board[i][n/2-1] == 'X') second_win = true;
    }
    else if (block_id == 4) {
```

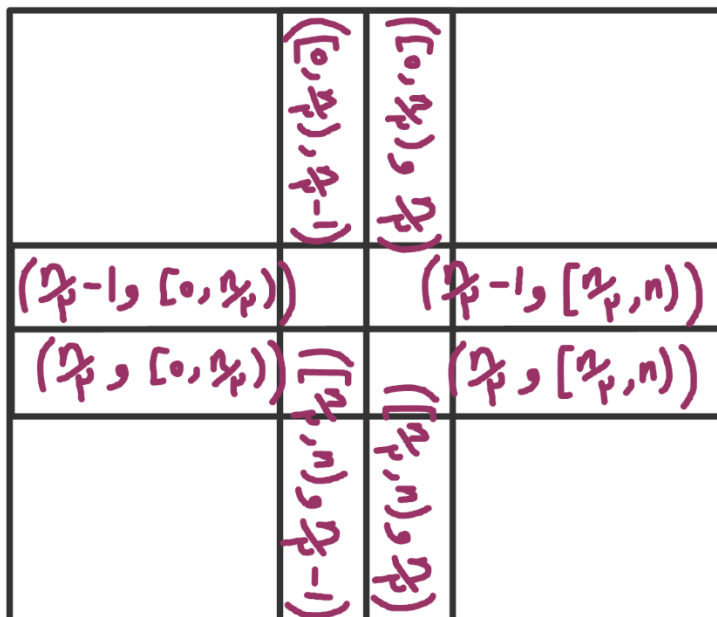
```

for (int j = n/2; j < n; j++) // top border
    if (check_win(board, n, n/2, j, board[n/2][j], k)) if(board[n/2][j] == 'O') first_win = true; else
    if(board[n/2][j] == 'X') second_win = true;

for (int i = n/2; i < n; i++) // left border
    if (check_win(board, n, i, n/2, board[i][n/2], k)) if(board[i][n/2] == 'O') first_win = true; else
    if(board[i][n/2] == 'X') second_win = true;
}

```

توضیح نموداری:



در این تابع صرفاً با بررسی 4 شرط، بازه مرز داخلی بلاک مورد نظر جنریت شده و خانه های آن به تابع check_win پاس داده می‌شوند. همچنین بازیکن یا بازیکنان برنده از طریق چرخش مشخص می‌شود.

تابع check_win_all:

در این تابع علاوه بر صدا زدن check_win_border، خانه ای که در آن مهره جدید گذاشته شده نیز چک می‌شود و از این پس، در صورت وقوع رویداد برد، برنده(ها) قطعاً مشخص شده اند.

```

void check_win_all(unsigned char **board, int n, int k, int block_id,
int last_x, int last_y, bool &first_win, bool &second_win) {
    char player = board[last_x][last_y];
    if (check_win(board, n, last_x, last_y, player, k)) {
        if (player == 'O') first_win = true;
        else if (player == 'X') second_win = true;
    }
    check_win_border(board, n, k, block_id, first_win, second_win);
}

```

آپشن ها و توابع کمکی

توابع save و load:

این دو تابع به سادگی با استفاده از جریان های ofstream و ifstream اطلاعات خواسته شده و لازم را در فایل تنظیمات ذخیره میکنند.

```
void save(unsigned char **board, int n, int k, int step, int min, int sec) {
    ofstream file("config.txt");
    file << n << " " << k << " " << step << "\n" << min << " " << sec << "\n";
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++)
            file << board[i][j] << " ";
        file << "\n";
    }
}

void load(unsigned char **&board, int &n, int &k, int &step,
atomic<int> &min, atomic<int> &sec) {
    ifstream file("config.txt");
    int m, s;
    file >> n >> k >> step >> m >> s;
    min = m, sec = s;

    board = new unsigned char*[n];
    for(int i = 0; i < n; i++) {
        board[i] = new unsigned char[n];
        for (int j = 0; j < n; j++)
            file >> board[i][j];
    }
}
```

تابع timer_thread:

این تابع تابعی است که به صورت شبه موازی (با نخ) اجرا میشود و تایمر بازی را هندل میکند. بعد از گذشت هر ثانیه، به متغیر secs یکی اضافه کرده و در صورت نیاز تبدیل ثانیه به دقیقه و ثانیه را انجام میدهد.

```

void timer_thread() {
    auto last = steady_clock::now();
    while (running) {
        auto now = steady_clock::now();
        if (duration_cast<seconds>(now - last).count() >= 1) {
            last = now;
            secs++;
            if (secs >= 60) {
                secs = 0;
                mins++;
            }
            cout << "\rTIME: " << setw(2) << setfill('0') << mins <<
            ":" << setw(2) << setfill('0') << secs << " | Input: " << flush;
        }
    }
}

```

و در ادامه در main چنین فعال میشود.

```

thread t(timer_thread);

```

توابع print_board و clearScreen و refresh:

این توابع فاقد ارزش الگوریتمی بوده و صرفاً قرار است محتوا را به طرز قابل قبولی به کاربر نمایش دهند. مثلاً وظیفه اصلی print_board نمایش دادن ماتریس صفحه بازی است.

رویداد برنده شدن:

در انتهای هر مرحله رویداد برد چنین بررسی میشود:

```

bool first_win = false, second_win = false;
check_win_all(board, n, k, block_id, x, y, first_win, second_win);

if(first_win && second_win) {
    cout << GREEN << "DRAW!\n" << RESET;
    break;
}
if (first_win) {
    cout << GREEN << "PLAYER 1 WINS!\n" << RESET;
    break;
}
if (second_win) {
    cout << GREEN << "PLAYER 2 WINS!\n" << RESET;
    break;
}

```

```

}
if (++step >= n * n) {
    cout << GREEN << "DRAW!\n" << RESET;
    break;
}

```

در انتهای برنامه علاوه بر توقف نخ تایمر، حافظه های تخصیص یافته شده نیز آزاد میشوند تا از نشت حافظه جلوگیری شود.

```

running = false;
t.join();

for(int i = 0; i < n; i++)
    delete[] board[i];
delete[] board;

```

تبلیغات:

الکتروگیگ، چنل جدید برای geek های تکنولوژی عه 🧐👓

شما به جمع دوستانه و مستقل خوره های تکنولوژی دعوت شده اید 🙌

<https://t.me/ElectroGeek255>

@ElectroGeek255

پرسپکتیو محتوایی: قراره از الگوریتم و دیتاستراچر پایه شروع شه (کنار BP و AP بدرد بخوره 🧐)،
 بعدا یه دوری میزنیم توی پایتون و بقیه زبان ها 🙌، وقتی نیاز شد هم میریم سراغ متلب 🧐، در این
 بین با آردوینو و در نهایت IOT که عشق خودمه هم آشنا میشیم 🧐 (مباحث دیگه مباحث طراحی مدار و
 ... هم الان یکم گنده گویی عه ولی چرا که نه)

@BP_with_mmd , @ElectroGeek255 , @mmd1023